



MEGACARTS ATTACKS

SYSTEM DOCUMENTATION

**SUBMITTED BY,
CHARITY NJERI WAWERU
DCF-01-0111/2025**

**A SYSTEM DOCUMENTATION SUBMITTED IN PARTIAL FULFILMENT FOR THE
AWARD OF DIPLOMA IN ?????????????????????????????? BY ZETECH UNIVERSITY**

APRIL 2025

DECLARATION

Leave this section blank until you've done the rest of the document

A paragraph saying that the project has been done by you, has not been outsourced from somewhere, and that this documentation has also been prepared by you, then leave a section like below for signing.

Student Name: _____ Sign: _____ Date:

Supervisor Name: _____ Sign: _____ Date:

DEDICATION

Leave this section blank until you've done the rest of the document

Say who you want to dedicate your project to (could be a parent, a loved one, or a mentor)

ABSTRACT

Leave this section blank until you've done the rest of the document

A summary of the project (how long it took, what tools/technologies/programming languages/frameworks/database technologies you've used. For what purpose has the project been built – to solve what problem and what you have achieved.)

DEFINITION OF KEY TERMS

Leave this section blank until you've done the rest of the document

Each key word you've used in this document and what it means, listed in alphabetical order e.g.

Database: a central repository for data

Webpage: a single page containing particular information of the website e.g. a contact page, built to be accessed from a browser.

ABBREVIATIONS AND ACRONYMS

Leave this section blank until you've done the rest of the document

Here, list down all short forms you've used in this document, with their meaning in full, and arranged in alphabetical order e.g.

HTML – Hyper Text Mark Up Language

SQL – Structured Query Language

(

LIST OF FIGURES

Leave this section blank until you've done the rest of the document

Here, list down all the image captions you have in your document, and in the correct order in which they appear in the document, from start to end e.g.

Fig 3.1-1 Login Page

Fig 3.1-2 Sign Up Form

Fig 3.2 Cart Management Code

LIST OF TABLES

Leave this section blank until you've done the rest of the document

Here, list down all the table captions you have in your document, and in the correct order in which they appear in the document, from start to end e.g.

Table 1.2 Functional Requirements

Table 1.3 Budget Breakdown

TABLE OF CONTENTS

Leave this section blank until you've done the rest of the document

Generate a table of contents for your document. Here is a quick guide on how to do that: <https://www.youtube.com/watch?v=0cN-JX6HP7c&t=87s>

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS (WORKPLAN)

1.1 Statement of Problem

It's become normal for people to shop online and enter sensitive personal and financial information — like credit card numbers, names and addresses — into e-commerce checkout pages on a regular basis. A lot of small to medium-sized online shops built on custom PHP platforms are being silently targeted by a form of cybercrime called Magecart attacks, in which criminals silently inject malicious code into the checkout page of a shop. This secret hidden code steals customers' payment details in the background as they type, without the shop owner or customer ever knowing. The stolen information is then sold or used for fraudulent transactions, leading to direct financial loss to customers and severe reputational damage to the compromised businesses. Shop owners, many of whom are not technical experts, have no reliable way of knowing when their checkout pages have been tampered with, leaving their customers exposed for days, weeks, or even months before the breach is ever discovered.

1.2 Study Justification

In this project we build Magecart Attack Detection System to counter the problem of invisible malicious script injection on PHP e-commerce checkout pages. The system solves this problem with three core features. First, it does Script Integrity Monitoring that automatically checks if the scripts running on a checkout page are the original allowed ones or secretly replaced/altered by an attacker. Second, it provides a Real-Time Alert System which immediately notifies the shop owner by email or SMS the moment a suspicious change is detected, thereby significantly shortening the time window of risk for the customers. Third, the shop owner can use a Security Dashboard to see the current status of all the scripts on their checkout page, review past alerts and act on them, all without needing deep technical knowledge. These features combined provide small PHP e-commerce businesses with a practical, affordable and accessible tool to safeguard their customers' payment data

1.3 System Objectives

1.3.1 General Objective

To develop a Magecart attack detection system for php e-commerce checkout page that monitors script integrity and alerts shop owners of unauthorized changes in real time

1.3.2 Specific Objectives

- (i) To implement script integrity verification that detects unauthorized modification
- (ii) To implement real time alert system that notifies the shop owner on email or SMS when a potential attack is detected
- (iii) To develop a security dashboard that displays the status of checkout pages logs and scripts on all detected security events
- (iv) To simulate an attack scene in a controlled environment as a way to test and validate the effectiveness of the system

1.4 Functional Requirements

User	User Activities	Features
Shop owner / shop admin	Log into the security dashboard	Secure login credentials and authentication
Shop owner / shop admin	View all scripts running on the check out pages	This inventory shows all active scripts and approval status
Shop owner / shop admin	Approve legitimate scripts	Script approval marks known safe scripts
Shop owner / shop admin	Receive instant notifications on suspicious scripts	Real time SMS or emails alert when system is triggered by unauthorized script changes
Shop owner / shop admin	View log of past security events	Security logs showing date and time, affected script and action taken
Shop owner / shop admin	Manually trigger a script integrity check	On demand scan button on the dashboard helps check check out page script immediately
Shop owner / shop admin	Disable or flag a suspicious script	One click script disables directly from dashboard without touch of backend code
Customer	Browse products and proceed to checkout	Standard e-commerce check out page
Customer	Enter payment details safely	Check out page protected byt running the integrity check script
Customer	Complete purchase and data is safe	System will detect and alert admin before a malicious script can access customer's data

Table 1.4 Functional Requirements Table

1.5 Breakdown of Tools & Resources to Be Used

Category	tool/resource	purpose
Front end	HTML/CSS	Build a stimulated e-commerce check out

		page
Backend	PHP	Core system logic
Database	MySQL	Store approved script hashes, security logs and alerts records
Local Server	XAMPP	Run php and MySQL locally
Alert system	PHPMailer	Send email notifications
Code editor	VS Code	Write and manage project code
Browser	Goggle chrome	Test and demo project
Version control	Github	Store and track project code

1.6 Project Schedule Breakdown

WEEKS	PROJECT MILESTONES					
	Project Planning & Analysis (System Documentation: Coverpage & Chapter One)	Project Design & Modeling (System Documentation Chapter Two)	Project Development & Testing (System Documentation Chapter Three)	Project Deployment (System Documentation Chapter Three)	Final Touches of System Documentation (Preliminary Pages, Chapter Four & References)	Project Presentation
5-9 May						
12-16 May						
19-23 May						
26-30 May						
2-6 June						
9-13 June						
16-20 June						
23-27 June						
30-4 July						
7-11 July						

14-18 July						
21-25 July						
11 Aug						

Table 1.6 Project Schedule Breakdown

Blue initially means that milestone has not achieved. Once you achieve that milestone, fill it with any other color of your choice to mean you're done with that part. That helps you to monitor your own progress versus the time scheduled.

When printing, this page must be printed in LANDSCAPE as the rest is printed in portrait

Transfer all the diagrams you used to design your system during milestone 3, to this chapter. Explain each diagram, have the diagram below the explanation, and a caption below the diagram

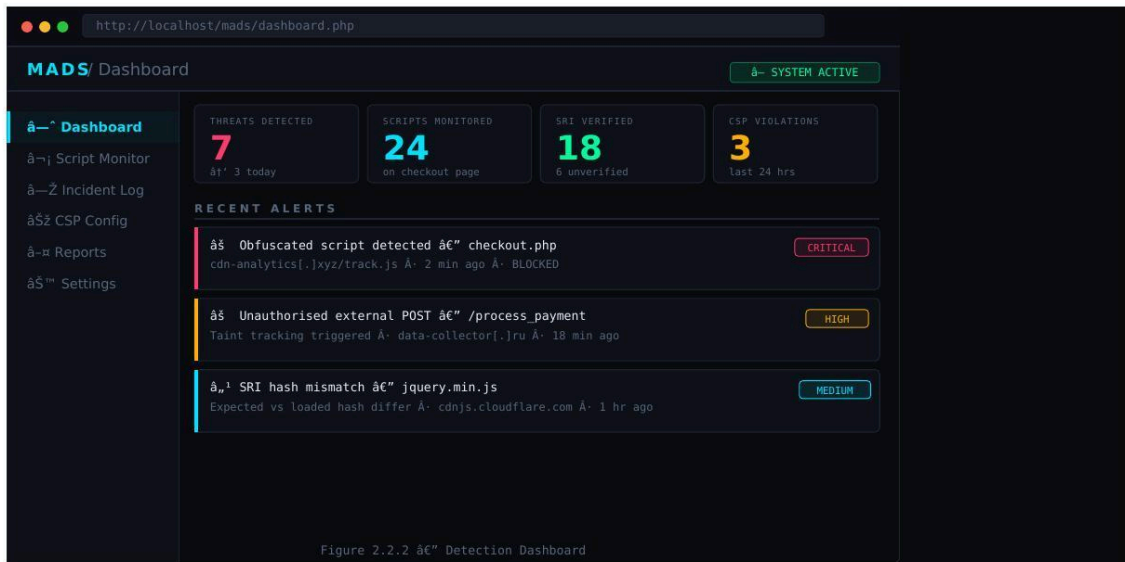
This chapter will present the system designs and models developed to visualize the project Megacart attack detection system, before the actualization. The diagrams will illustrate how the system will detect malicious scripts, how the ecommerce page is monitored, and how data flows through detection engine and alert pipeline.

The following diagrams will illustrate the user interface screens that the system admin will interact with when using the Megacart Attack Detection System.

The secure entry point to MAD system, displays a login card reinforcing the security focused nature of application. It also has authenticate button and a footer that show only authorised persons are allowed.

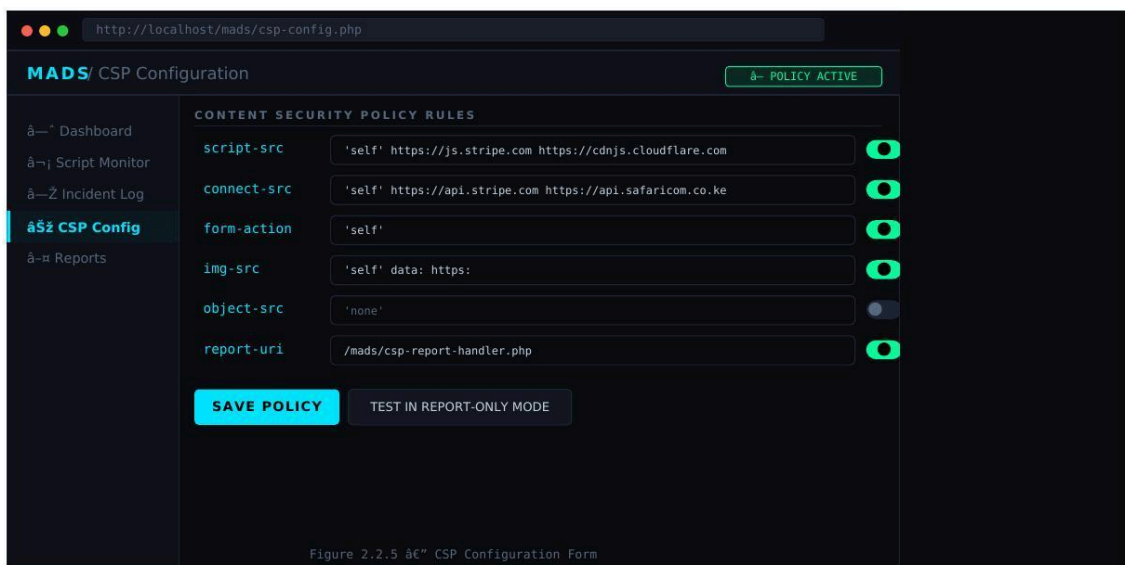
2.2.2 Detection dashboard

This is the main screen that is displayed on successfully login. It shows the other system functions and also summary cards showing incidents detected and recent alert panels which have the threat description , source url, timestamps, and color coded severity badges.



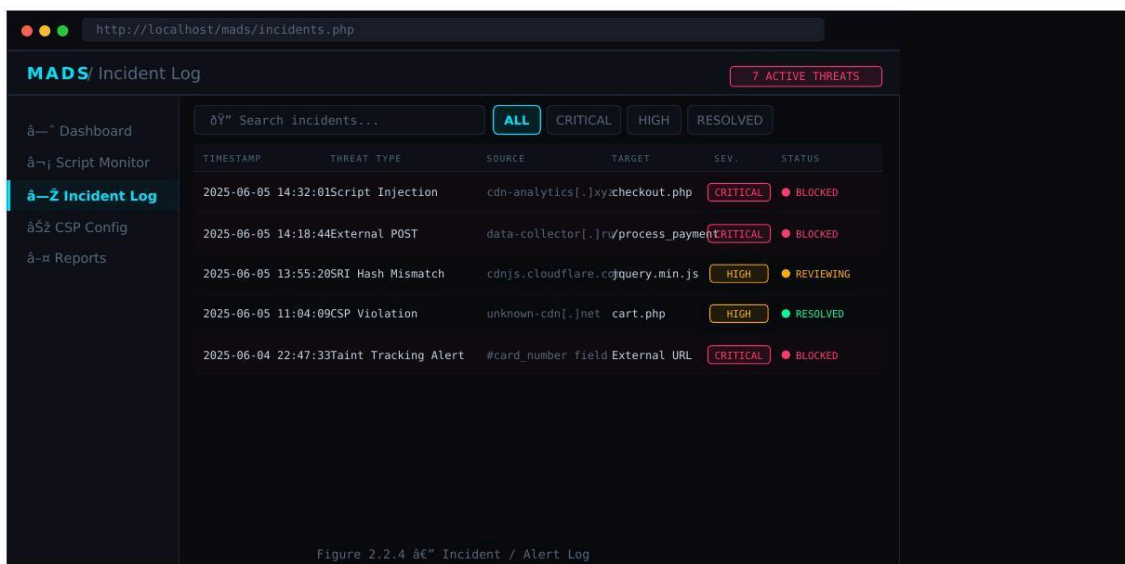
2.2.3 Script Monitoring

A comprehensive table of all javascript files loaded on the checkout pages. Each row with its script file names, its SRI hash value, original category, verification indicator, and action link, color coded status



2.2.4 Incident Logs

Incident logs is the record keeping screen of the system. Has a search bay that allows admin to filter the contents for viewing



2.2.5 CSP configuration

Allows the admin to define and manage the CSP rules that govern external sources the browser is permitted to load scripts and resources from. Has a button to toggle on each row to enable or disable taht is color coded

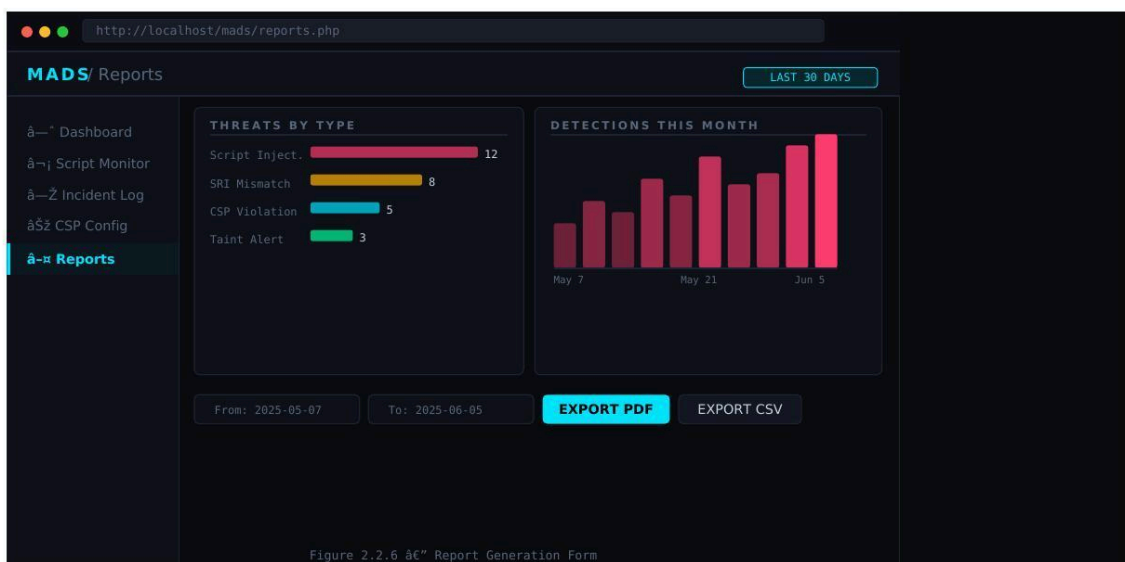
The screenshot shows the 'MADS/ CSP Configuration' page. On the left is a sidebar with navigation links: Dashboard, Script Monitor, Incident Log, CSP Config (highlighted), and Reports. The main content area is titled 'CONTENT SECURITY POLICY RULES'. It contains a table of rules with input fields and toggle switches. The rules are: script-src, connect-src, form-action, img-src, object-src, and report-uri. Each rule has a corresponding toggle switch on the right, which is either green (on) or grey (off). Below the table are two buttons: 'SAVE POLICY' and 'TEST IN REPORT-ONLY MODE'. A status bar at the top right indicates 'POLICY ACTIVE'.

Rule	Value	Status
script-src	'self' https://js.stripe.com https://cdnjs.cloudflare.com	On
connect-src	'self' https://api.stripe.com https://api.safaricom.co.ke	On
form-action	'self'	On
img-src	'self' data: https:	On
object-src	'none'	Off
report-uri	/mads/csp-report-handler.php	On

Figure 2.2.5 "CSP Configuration Form"

2.2.6 Report generation

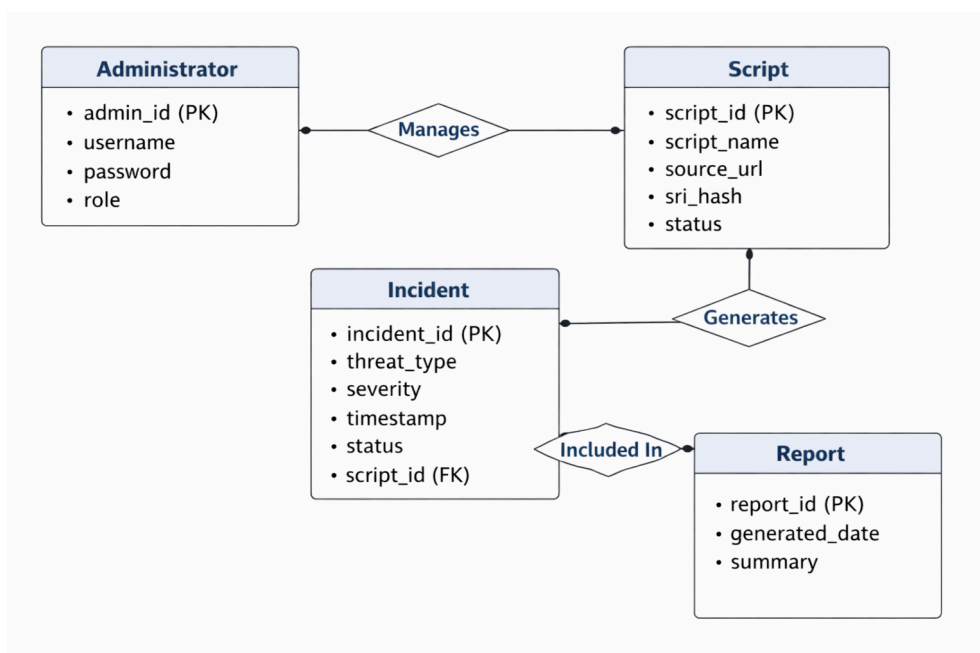
Provides visual analytics and export functionality for the admin. Has side by side visualization panels



2.3 Logic Models

2.3.1 logic process flowchart

This flowchart illustrates the authentication procedure for the admin accessing the Megacart Attack Detection System. Here the admin inputs in the username and password to login, system validates against the database, if incorrect an error message is displayed, if correct access is granted



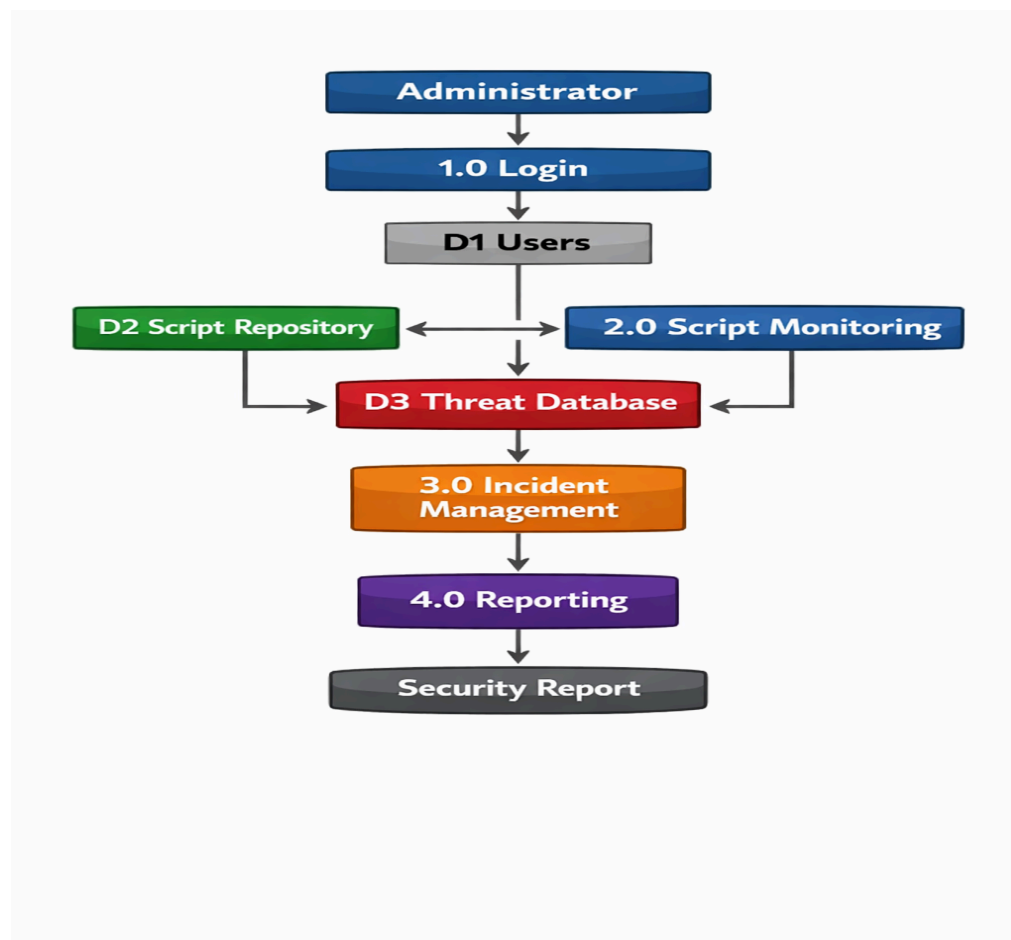
2.3.2 System Flowchart

System flowchart shows overall operation of Megacart Attack Detection System. It illustrates from when an admin login , where if credentials are correct they are granted access and a Dashboard is displayed, showing system functions. An admin can monitor the scripts, review detected threats, configure content security policy settings, investigate incidents and generate incident reports. Continuously, the system scans the website resources, validates script integrity through SRI checks, detect suspicious activities and record security events



2.3.3 Data Flow

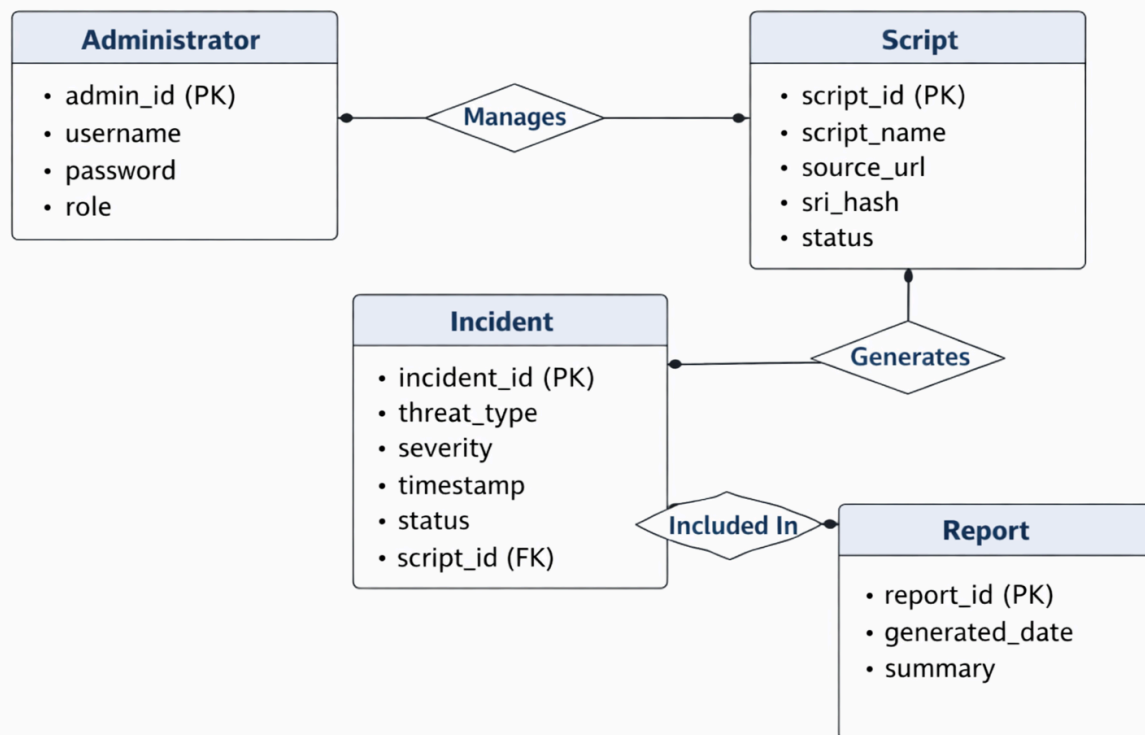
This chart shows how data moves around in the system.. Admin interacts with the system when they work on the system functions. Login process communicates with the user database for authentication. Script monitoring exchanges information with the script repository and threate database to analyze website scripts and identify malicious behaviour. Incidend management process stores and retrieves security alerts while reporting process gathers data and generates security reports for admin, it ensures effective threate detection, analysis and reporting.



2.3.4 Entity Relationship Diagram

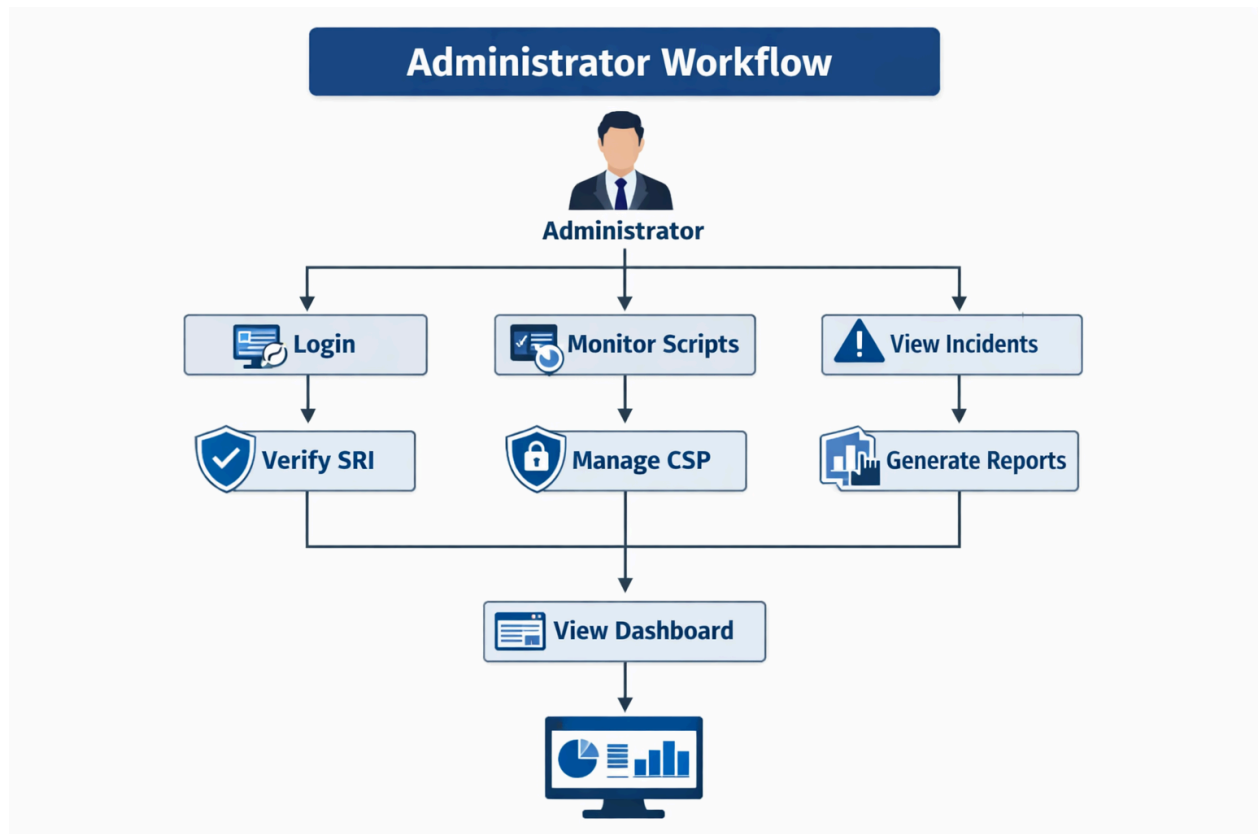
This diagram represents the database structure of the MAD system. Consists of User, Script, Incident, and Checkout page. The user entity stores admin credentials. Script entity contains details of java scripts file which include source url, SRI expected hash value, origin category and verification status. Incident entity records all detected events, including timestamp, threat type, target page, severity level and resolution status. Checkout page entity tracks which scripts are loaded and when the last scan happened.

The relationship between these entities is that one admin can manage many scripts; one script can trigger many incidents; and the checkout page can load many scripts. Foreign keys link incidents back to their triggering scripts, ensuring data integrity and traceability.



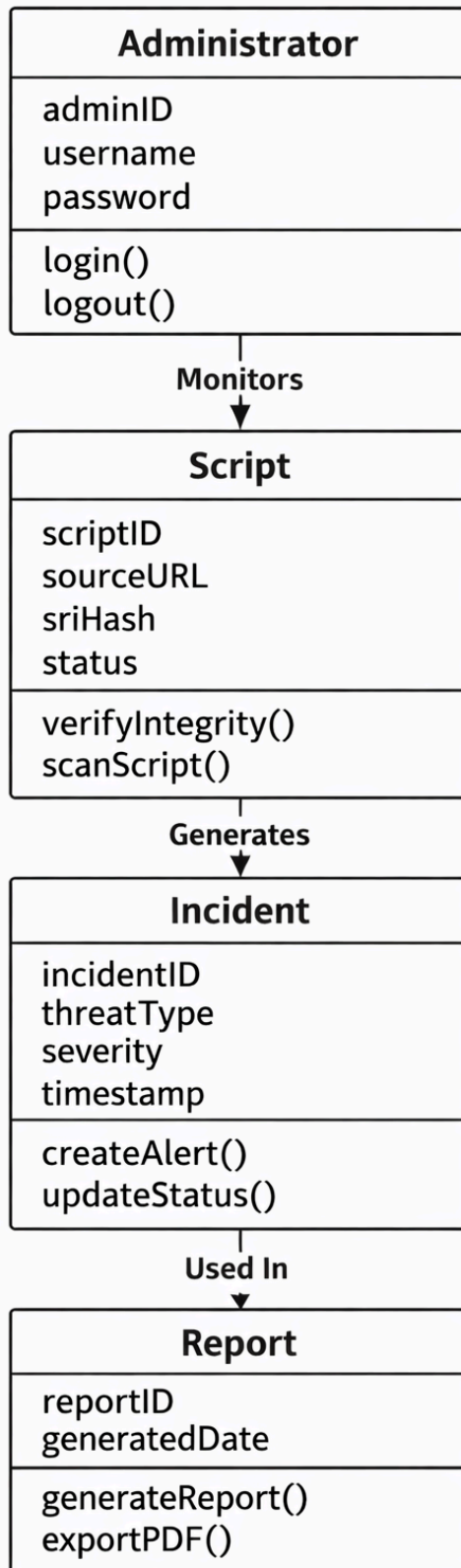
2.3.5 Use case diagram

Shows the interactions between the admin and MAD system. Admin is the primary actor and they can perform the following, login to the system, view detection dashboard,, monitor javascripts scripts on checkout page, review and filter incident logs, configure CSP rules, whitelist or blackmail script.



2.3.6 Class diagram

This diagram shows main classes in MAD system and how they relate with each other.



CHAPTER THREE: SYSTEM IMPLEMENTATION (DEVELOPMENT, TESTING AND DEPLOYMENT)

This is where you report on your development journey e.g. from when you downloaded your IDEs, and coding, to implementing your database to deploying your system

3.1 Introduction

A paragraph telling the reader what they expect to see in this chapter – a journey of your development

3.2 User Interface Development

Your system has two sides – the user side (where different users can interact with the system) and the logic side.

Under this section you will screenshot all the User Interfaces you have built (login pages, sign up forms, report page, dashboard, etc.) and the code for them if you built them from scratch, e.g. a screenshot of the html and css code for sign up form.

Organize them into subheadings e.g. 3.2.1 Login Page Development (under this section, a brief explanation of what it's been built for, a screenshot of the page and a caption underneath it, screenshots of its html and css code used to develop it, and captions underneath every screenshot)

3.3 Logic Development

Your system has two sides – the user side (where different users can interact with the system) and the logic side. The logic side is how the system is performing activities using data e.g. login validation, processing checkout and generating a receipt, storing data into database, retrieving products from DB and showing them on a certain page. This is called logic.

*Under this section you will screenshot all the **Codes** or **Simulations** that handle different processes/activities. Organize them into subheadings e.g. 3.3.1 Login Validation Code (herein, explain in brief how the system handles login, take screenshots of the login validation code, and caption it)*

3.4 Testing

Detail out what actions you took to test whether different features of your system are working. And corrections you made to different parts to ensure that they are working as planned.

3.5 Deployment

Mention on what platform have you deployed your system (e.g. if it's a website, what web hosting platform did you use – and copy paste a link to your website, if it's an android application – the process of putting up your app on Google Playstore.). Essentially, detail out the process of how you transformed your project into a form that can be run/installed by anybody.

CHAPTER FOUR: CONCLUSION AND RECOMMENDATION

4.1 Conclusion

A paragraph summarizing chapter one, two and three. And finish by telling the reader what new things you learnt during the entire time developing your project, and challenges faced.

4.2 Recommendation

Here, speak as if addressing another developer of the same project you've been working on. Suggest to him/her things that they can do to improve the system if they so choose to pick it up and enhance it. E.g. if you created an e-receipt for customers, suggest to him/her to consider integrating a physical receipt printer since some customers would prefer that and also you couldn't implement that because you didn't have the budget for a printer. If your system is a staff clock-in-and-out system and in your system a staff has to type in their staff number into the system, suggest to the developer what better technologies he/she can use to enhance that specific part e.g. biometric or swipe-card, maybe since you did not have the budget to buy those devices and/or didn't have the technical skills for that. Or if for payment your system is using Pay-After-Delivery, suggest to him/her to consider integrating with MPESA or Bank Card for pre-delivery payments.

NB: Recommendation is not an excuse for a NOT WORKING system, e.g. it is not saying that your system can't store any data because you didn't have the time to implement a database, No. It is a suggestion for how your ALREADY WORKING SYSTEM (with the lower technologies you've used) can be MADE BETTER, with HIGHER/BETTER TECHNOLOGIES. E.g. you used MySQL to store structured data, suggest for them to use a NO-SQL approach to store non-structured data like customer feedback and product reviews.

REFERENCES

A list of all the websites, articles, books, YouTube channels, AI Conversations (copy link address) that helped you during your project.

Format them in APA7 format, in alphabetical order.

A guide on referencing from web articles/tutorials:

<https://libraryguides.vu.edu.au/apa-referencing/7Webpages> (see in-black-box)

A guide on referencing from social media articles:

<https://libraryguides.vu.edu.au/apa-referencing/7SocialMedia> (see in-black-box)

A guide on referencing from conversations you had with AI e.g. ChatGPT:

<https://libraryguides.vu.edu.au/apa-referencing/generativeAI> (scroll to end of page)

Done with references, now go back to PRELIMINARY PAGES (pages before chapter one), and fill those pages